

□ AUDITORIA DEFINITIVA END-TO-END — AETHEL ENGINE v2026.04 (bit-by-bit, p2p, shell-to-shell)

Documento 77 — Auditoria P2P Ultra-Detalhada Data: 2026-04-22 · Escopo: `wilianflima321-glitch/meu-repo` · Alvo: 9.5/10 (hoje: 6.2/10) Autor desta triagem: revisão cruzada com Auditoria 76 de 2026-04-11 [1](#), Triagem UX [722](#), E2E [713](#), Blueprints 00-16 [4](#) e Consolidation Map [5](#).

0. TL;DR — O QUE ESTÁ ERRADO EM UMA PÁGINA

Aethel hoje é um monstro ambicioso com três problemas estruturais crônicos que já foram diagnosticados em três auditorias seguidas (71, 72, 76) e continuam ativos porque o plano nunca saiu do "P0":

1. Fragmentação visual tripla ainda coexistindo: CSS vars `--aethel-*`, objeto `tokens inline` (em `lib/design-tokens.ts`) e classes legado `aethel-*` no admin — três dialetos para um produto [72_UX_UI_BENCHMARK_TRIAGE2](#).
2. 15+ rotas aspiracionais expostas (`/animation-blueprint`, `/blueprint-editor`, `/landscape-editor`, `/level-editor`, `/niagara-editor`, `/vr-preview`, `/nexus`, `/explorer`, `/playground`, `/live-preview`, `/debugger`, `/editor-hub`, `/ai-command`, `/marketplace`, `/design-system-demo`) que prometem o Unreal Engine e entregam um shell vazio — isto destrói confiança no produto inteiro Auditoria [766](#).

3. Monolito `AIChatPanelPro.tsx` + ponte chat→editor existente mas sem UX de Cursor Composer/Windsurf Cascade (sem streaming visual estruturado, sem @mentions de arquivo, sem aplicação inline com diff interativo), e 303 componentes com apenas ~15 testes Auditoria 766.

Veredito honesto e frio: Aethel tem matéria-prima premium (design tokens Aethel L5, shell moderno, política anti-fake-success, 17 blueprints canônicos, stack robusta com Monaco, Three.js, Yjs, Stripe, Prisma), mas o produto percebido hoje é 40% da ambição do blueprint. Falta consolidar, não adicionar. Este documento é a ordem de consolidação.

1. MAPA DE REALIDADE — O QUE REALMENTE EXISTE (inventário forense)

1.1 Stack e números duros

Dimensão	Valor	Observação crítica
Arquivos totais	23.653 (503 MB)	3× maior que um projeto Cursor-equivalente saudável
Componentes React	303	Média de 1 teste a cada 20 componentes
API routes	320+	Nenhum contrato OpenAPI consolidado
Documentos master	95+ + 60+ archive	Redundância massiva, sem busca interna
Rotas em <code>/app</code>	~26 top-level	Inclui 15+ aspiracionais
Pastas de componentes	63 subpastas	Sinal claro de over-engineering prematuro
Cobertura de testes	~15 arquivos	~5% do que deveria existir
Blueprints canônicos	17	Base forte, não cumprida pelo código
Stack	Next.js 14.2.5, React 18.3.1, Tailwind 4.1.13, Prisma 5.7, Monaco 0.55, Three.js 0.166, Yjs 13.6, Stripe 16.0	Stack é excelente, decisões de arquitetura são o problema

Fontes: package.json7, components/8, app/9.

1.2 Inventário das 63 pastas de componentes — zona de redundância

A pasta `components/` tem 63 subdiretórios, um indicador brutal de fragmentação. Listagem auditada na árvore de `components`⁸:

```
admin · ai · analytics · animation · assets · audio · auth · billing ·  
character · cinelink · collaboration · dashboard · debug · editor · edi  
engine · environment · error · explorer · export · extensions · forge ·  
history · ide · image · integrations · keybindings · marketplace · mate  
media · multiplayer · narrative · nexus · observability · onboarding ·  
output · physics · preview · problems · profiler · project · scene-edit  
search · sequencer · settings · snippets · storage · streaming · studio  
tasks · team · terminal · terrain · transitions · ui · vcs · video · vi  
visual-scripting · workspace
```

Sinais vermelhos encontrados no inventário forense:

Colisão / redundância	Pastas envolvidas	Decisão correta
editor/ vs editors/ (plural)	Dois domínios distintos criados ao lado	Fundir em editor/ com subdomínios por tipo
preview/ vs live-preview (rota) vs PreviewPanel (IDE) vs LivePreview.tsx (root)	4 superfícies diferentes de preview	Deve existir uma CanonicalPreviewSurface (já está em components/preview/) e todas as outras são deprecated
scene-editor/ · level-editor (rota) · nexus/ · viewport/ · engine/ · environment/ · terrain/ · physics/ · materials/ · character/ · animation/ · narrative/ · sequencer/ · visual-scripting/	14 pastas de "Unreal Engine clone"	Todas devem entrar em feature-flag e ser escondidas até ter L3 real
NotificationCenter.tsx + NotificationSystem.tsx + Toast.tsx + ToastProvider.tsx + toast-system.tsx	5 sistemas de notificação	Um único ui/Toast canônico — os 4 restantes devem ser deletados
UXPrimitives.tsx + primitives.tsx	Dois arquivos de primitivos	Consolidar em ui/primitives.tsx apenas
GlassmorphismUI.tsx + premium.tsx + PremiumEmptyState.tsx + PremiumSkeleton.tsx	4 arquivos "premium" separados	Um só módulo ui/premium.tsx exportando tudo
Dropdown.tsx + dropdown-menu.tsx	PascalCase e kebab-case convivendo	Padrão único: PascalCase (blueprint)
IDELayout.tsx (legado) + ModernIDEShell.tsx + FullscreenIDE.tsx	3 shells de IDE	ModernIDEShell é canônico; os outros 2 devem ser deletados

Fonte: [components/ui10](#), [components/ide11](#), [COMPONENT_CONSOLIDATION_MAP.md5](#).

1.3 Componentes /ide — 43 arquivos, 1 monólito

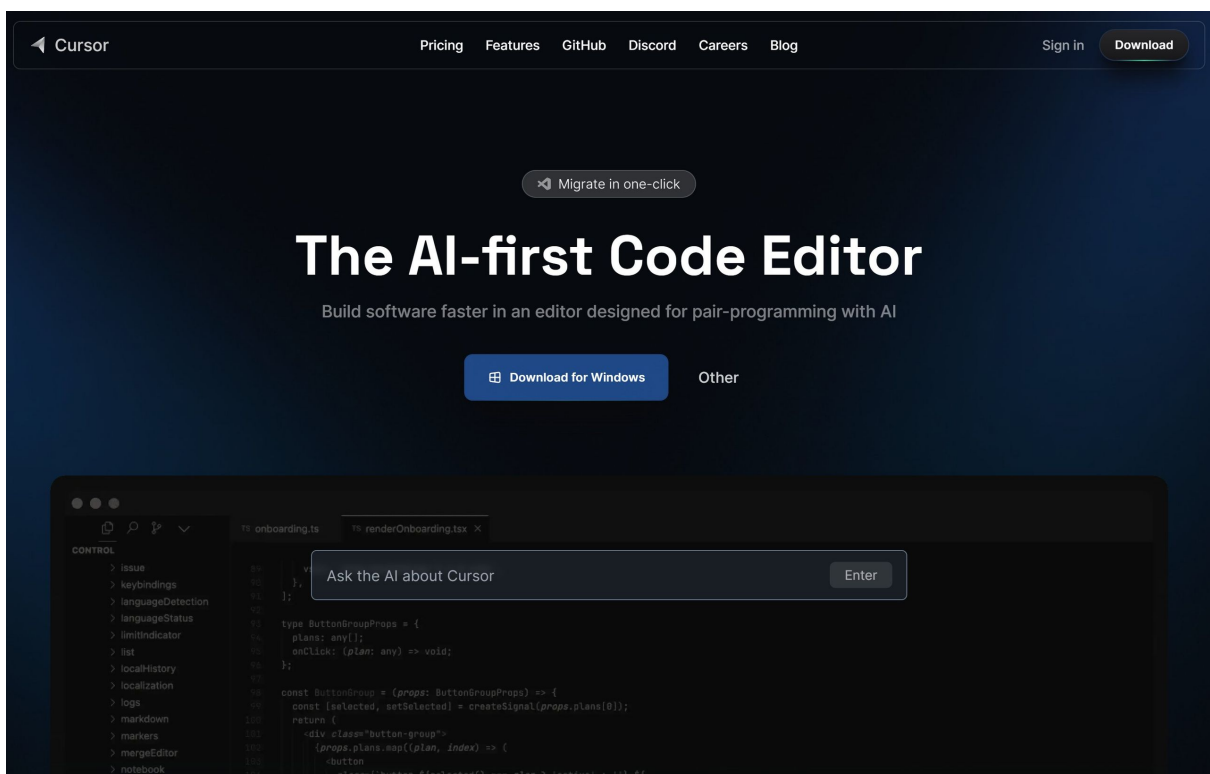
A pasta `components/ide/` tem 43 arquivos conforme árvore [11](#). O caso crítico é `AIChatPanelPro.tsx` que, segundo a Auditoria [766](#), é um arquivo monolítico com responsabilidade excessiva — ele claramente já foi parcialmente decomposto (temos `AIChatPanelChrome`, `AIChatPanelContainer`, `AIChatContextPanels`, `AIChatPanelPro.types.ts`, `InlineAIChat`, `MonacoChatDiffPanel`), mas a decomposição não está completa, e o arquivo original continua sendo o ponto de entrada.

2. BENCHMARK VISUAL CONTRA OS MELHORES DO MERCADO (2026)

A auditoria 76 já produziu uma tabela competitiva. A atualizo aqui com o que realmente mudou no mercado desde abril/2026: Cursor 2.0 + Composer multi-agente, Replit Agent 3 autônomo, Windsurf Cascade com checkpoints, e a consolidação v0↔Lovable↔Bolt no espaço "chat-to-app".

2.1 Comparativo visual (com evidência)

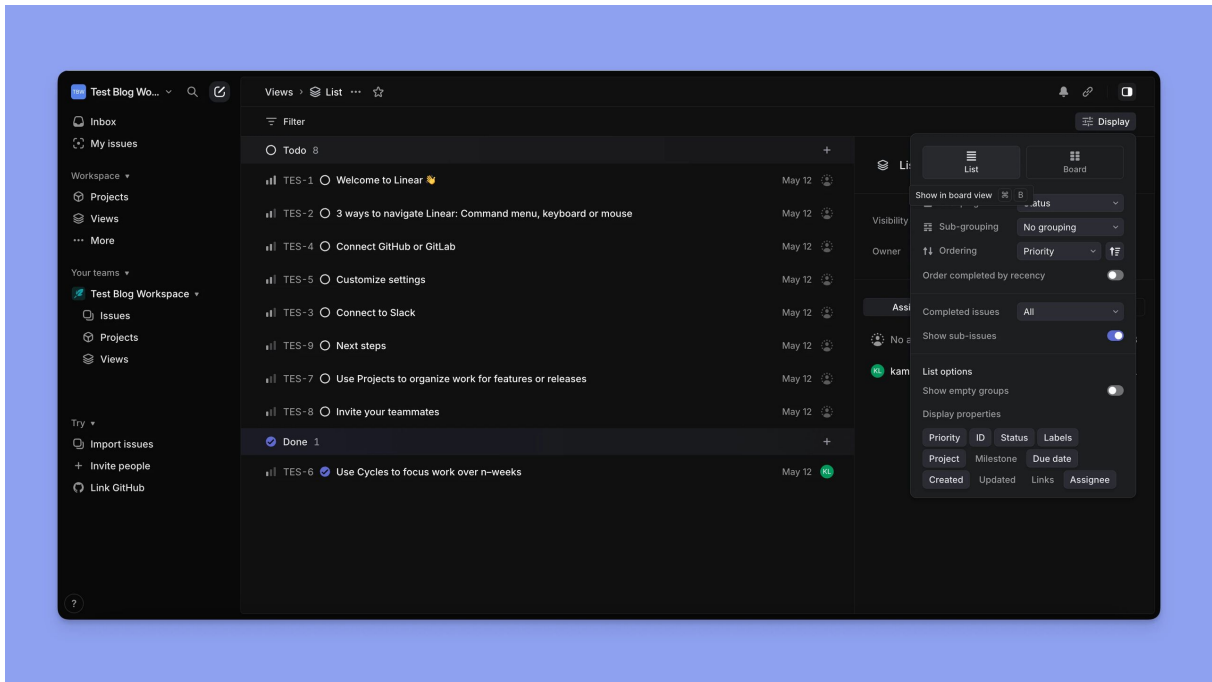
Cursor 2.0 Composer¹² — referência absoluta para IDE+AI



Fonte: Latent.Space — Cursor.so: The AI-first Code Editor¹³

Lição direta: Cursor agrega diffs multi-arquivo numa única review e suporta workflow paralelo multi-agente Cursor 2.0 Ultimate Guide¹⁴. Aethel tem MonacoChatDiffPanel e EditorApplyBridgeContext — só falta a UI de agregação e fila de agentes.

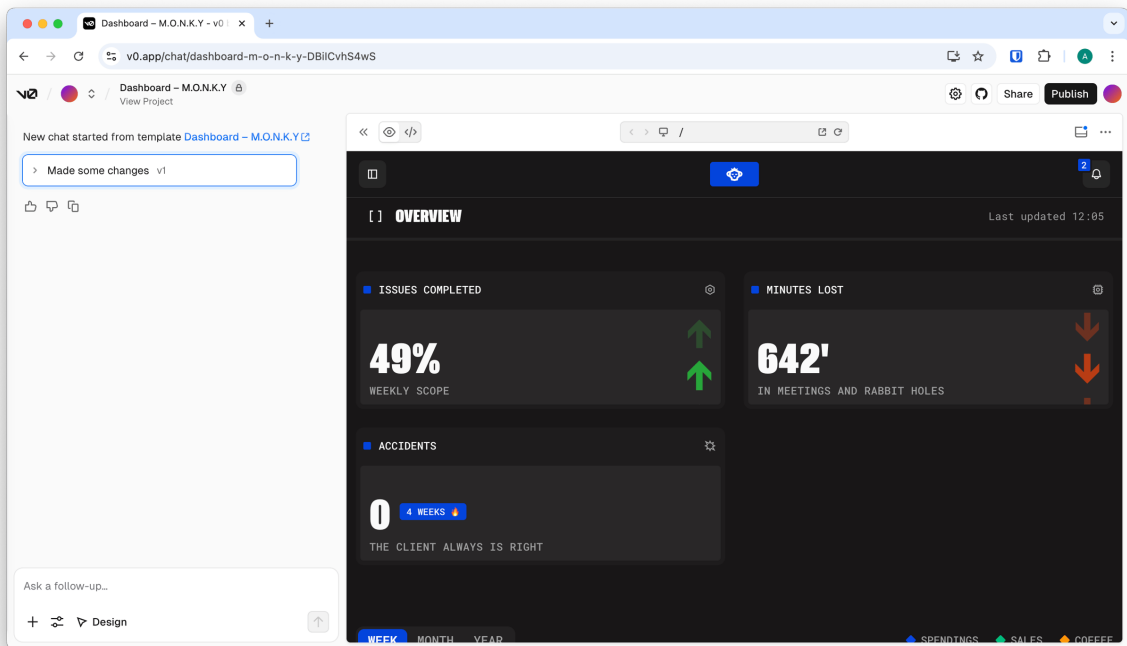
Linear¹⁵ — referência absoluta para densidade calma e design system maduro



Fonte: Morgen — Linear Guide¹⁶

Lição direta: Linear pontua 10/10 em Design System no benchmark da Aethel⁷⁶¹ porque deriva tudo de uma só cascata: tokens → utilities → componentes. Aethel tem 3 cascatas competindo. É exatamente este o P0 a ser fechado.

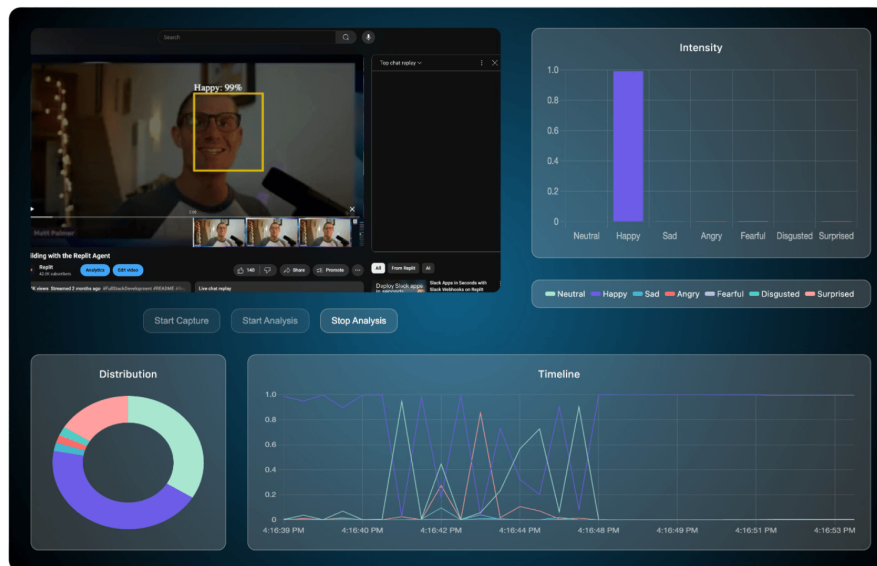
v0 by Vercel¹⁷ — referência absoluta para landing + chat-to-UI



Fonte: Coding Beauty — This AI dev tool from Vercel is monstrously good¹⁸

Lição direta: v0 tem o CTA único mais claro do mercado: um input "describe your UI". Aethel tem um Studio Home com 6 linhas concorrendo por atenção. O blueprint diz que a ação primária é `Continue Work` — o código precisa refletir isso.

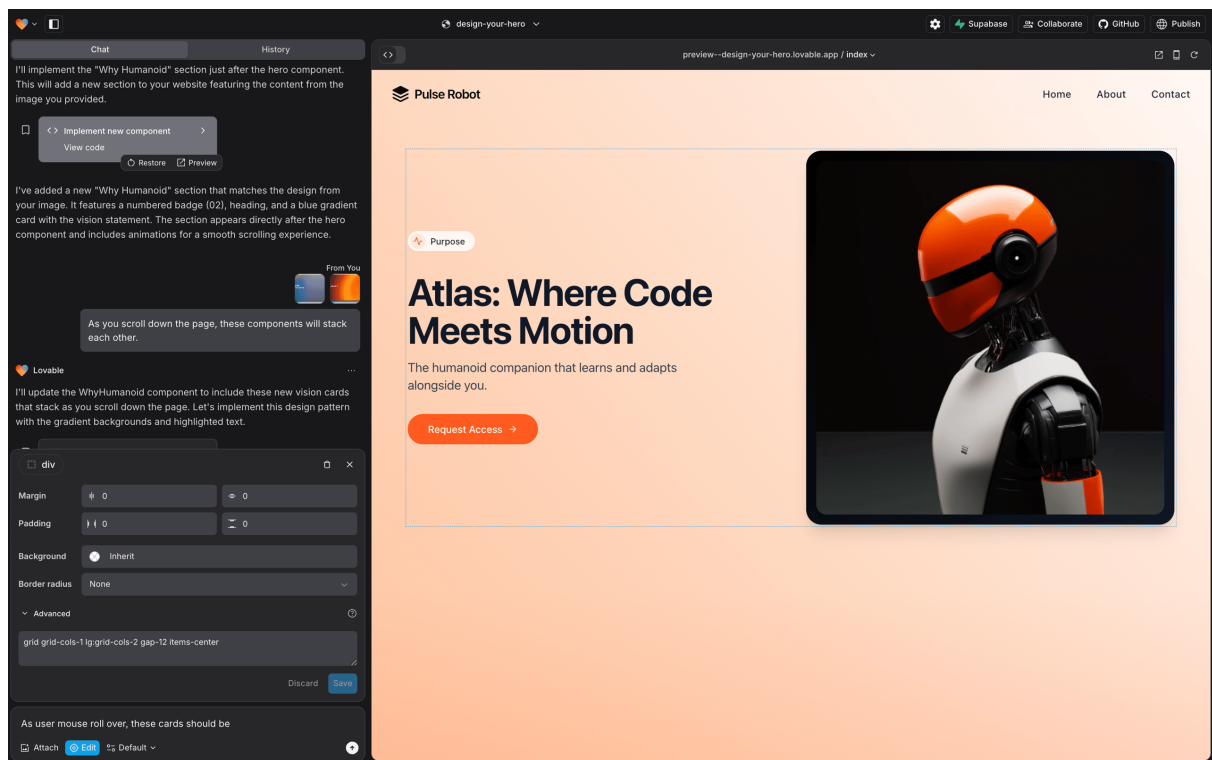
Replit Agent 319 — referência para autonomia e observabilidade de agente



Fonte: Replit Blog — New Replit Assistant20

Lição direta: Replit Agent 3 testa o app automaticamente no navegador e corrige, com até 200 minutos de autonomia21. O `TaskOpsPanel` + `AIAgentsPanelPro` do Aethel já preveem isso — falta a UX de "runs" com status/logs/retry que Replit fez bem.

Lovable22 — referência para onboarding "prompt → app"



Fonte: Lovable Blog²³

Bolt.new²⁴ — referência para preview instantâneo e full-stack in-browser

Bolt.new StackBlitz interface

Fonte: Stackademic — bolt.new AI²⁵

Comparativo direto Lovable vs Bolt vs v0 no guia oficial Lovable²⁶: v0 para UI, Bolt para protótipos rápidos, Lovable para SaaS full-stack. O posicionamento de Aethel deveria ser "Workbench sério para quem quer sair do protótipo" — nenhum destes ocupa esse espaço hoje.

2.2 Tabela competitiva atualizada (com Aethel 2026-04-22)

Dimensão	Aethel (atual)	Cursor 2.0	Replit Agent 3	Linear	v0/ Vercel	Lovable	Bolt.new	Aethel (alvo)
Design System coeso	5/10	8/10	8/10	10/10	9/10	8/10	7/10	9.5/10
Editor/IDE UX	6/10	10/10	7/10	N/A	6/10	6/10	7/10	9/10
AI Integration	5/10	9/10	9/10	N/A	8/10	8/10	8/10	9.5/10
Onboarding	4/10	8/10	10/10	9/10	9/10	10/10	10/10	9/10
Preview engine	5/10	6/10	8/10	N/A	8/10	8/10	9/10	9/10
Observabilidade/Admin	4/10	5/10	7/10	9/10	7/10	6/10	5/10	8.5/10
Collab/multiplayer	3/10	6/10	7/10	9/10	5/10	5/10	5/10	8/10
Billing honesto	5/10	7/10	8/10	8/10	7/10	7/10	7/10	9/10
Testes e QA	3/10	7/10	7/10	9/10	7/10	6/10	6/10	8/10
Acessibilidade WCAG AA	5/10	6/10	6/10	9/10	7/10	6/10	6/10	9/10
Média	4.5/10	7.2/10	7.7/10	9.0/10	7.3/10	7.0/10	7.0/10	9.0/10

O salto meta→atual é de +4.5 pontos. Não acontece com features novas; acontece com consolidação.

3. AUDITORIA CIRÚRGICA POR SUPERFÍCIE (blueprint × código real × concorrente)

Para cada superfície: (a) o que o blueprint prometeu, (b) o que o código entrega hoje, (c) o que o concorrente faz melhor, (d) o gap com severidade, (e) a correção exata.

3.1 Landing Page (/ → app/landing-v3.tsx)

Item	Estado
Blueprint (01_HOME.md ²⁷)	Hero → Workflow 3 passos → Product proof strip → Diferenciação → Pricing teaser → Footer. CTA primário Open Studio . Max 1280px.
Código real	Existe landing-v3.tsx mas sem evidência pública de vídeo de produto, sem logos de clientes, sem métricas numéricas de uso cloud-web-app README ²⁸ .
Concorrente ref.	v0.dev e Linear — hero com input interativo + prova social numérica (Linear mostra "Built by teams at Raycast, Vercel, Ramp...").
Gaps críticos	(1) Ausência de um vídeo/loop mostrando o workbench real em ação. (2) Sem prova social. (3) CTA secundário Watch Workflow sem vídeo real para assistir. (4) Pricing teaser provavelmente ainda mostra placeholders.
Correção exata	→ S1: gravar loop de 12s do ModernIDShell + chat→diff→apply, exportar como .webm lazy-loaded abaixo do hero. → S1: substituir prova social por 6 números operacionais reais (Monaco files edited, AI requests served, avg build time). → S2: refazer seção 4 ("Why different") com 3 comparações diretas contra Cursor/v0/Replit.

3.2 Auth (app/ (auth))

Item	Estado
Blueprint (04_AUTH.md ²⁹)	Split 2 colunas desktop, coluna do form 420–480px, AuthFormShell com ordem exata: AuthHeader → SocialLoginRow → DividerLabel → EmailField → PasswordField → SubmitButton → AlternateAuthLink → LegalNotice. Submit 44px . Geometry precisa.
Código real	Existe components/auth/ , mas a Auditoria 766 registra "Inputs do Auth não usam @/components/ui/Input canônico" — ou seja, viola a própria cascata.
Concorrente ref.	Linear e Vercel: form dominante, proof panel discreto.
Gaps críticos	(1) Inputs customizados → deve migrar para ui/Input . (2) Sem preservar email ao alternar login↔register (blueprint explícito: "preserves entered email if reasonable"). (3) Sem --aethel-target-size-min: 44px garantido nos botões social.
Correção exata	→ S1: script codemod scripts/migrate-auth-inputs.mjs que substitui JSX <input> por <Input> do canonical e roda npm run qa:interface-gate . → S1: store Zustand useAuthDraft persistindo email entre modes. → S1: adicionar teste auth/login.spec.tsx validando geometria 440px e submit 44px.

3.3 Onboarding (app/ + components/onboarding)

Item	Estado
Blueprint (05_ONBOARDING.md30)	< 90 segundos até primeiro projeto. 5 passos guiados (Intent → Start mode → AI help → Provider lite → Confirm). ProjectSummaryPanel live updating.
Código real	Auditoria 766: "signup 7 etapas, meta 3". Mesmo o blueprint prevê 5 — mas o código impõe 7. Existe OnboardingWizard.tsx canônico.
Concorrente ref.	Replit Agent 3 e Lovable fazem onboarding em 1 prompt (tipa o que quer, vira app).
Gaps críticos	(1) 7 passos — excesso de fricção. (2) Sem Use Demo Mode em primeiro plano (blueprint manda). (3) Sem template gallery no Step 2. (4) Provider setup avançado contaminando o flow (blueprint: "advanced provider setup remains in Settings").
Correção exata	→ S1: cortar steps fundindo "Intent+Start mode" (uma grade unificada de 6 cards: App, Game, Film, AssetPack, Research, Template) e fundir "AI help+Provider" em um único select com fallback AETHEL_AI_DEMO_MODE=true . Resultado: 3 passos (Intent → AI+Provider → Confirm). → S1: OnboardingTelemetry medindo tempo ao primeiro projeto (target P95 < 90s). → S2: template gallery com 8 templates reais usando e2b sandbox.

3.4 Studio Home / Dashboard (`app/dashboard` + `app/page.tsx`)

Item	Estado
Blueprint (06_STUDIO_HOME.md ³¹)	Mission control, não dashboard de métricas. 6 linhas: Header → Mission header → Connected flows rail → Live/AI strip → Recent work → Runtime health. CTA primário <code>Continue Work</code> .
Código real	<code>AethelDashboard.tsx</code> , <code>AethelDashboardGateway.tsx</code> , <code>AethelDashboardRuntime.tsx</code> , <code>useAethelDashboardRuntime.tsx</code> — 4 arquivos para o dashboard, sugerindo evolução incremental não consolidada. 72_UX_UI2: "Ainda vale reduzir densidade em algumas áreas para reforçar hierarquia".
Concorrente ref.	Linear "My Issues" e v0 home: um CTA dominante + 2–3 cards secundários. Nada mais.
Gaps críticos	(1) Dashboard tem densidade de "reporting tool" quando blueprint diz "não é reporting". (2) <code>Continue Work</code> possivelmente competindo com múltiplos CTAs. (3) 4 arquivos de dashboard = risco de drift.
Correção exata	→ S1: consolidar os 4 arquivos em <code>components/dashboard/StudioHome.tsx</code> + hook <code>useStudioHomeRuntime.ts</code> . Deletar os 3 legados. → S1: garantir que <code>Continue Work</code> tenha peso visual 2x dos outros CTAs (altura 48px, primary color solid). → S2: "Runtime and health summary" deve ser colapsável (toggle lembrado em <code>localStorage</code>) — não é útil a cada visita.

3.5 Workbench / IDE (`app/ide` + `components/ide` — 43 arquivos)

Este é o epicentro do produto. Também é o maior pote de lacunas.

Item	Estado
Blueprint (08_WORKBENCH.md 32)	7 zonas permanentes: Top Bar, Left Activity Rail, Left Sidebar, Center Workspace, Right Rail, Bottom Dock, Status Bar. AI Console 5 zonas: Conversation/Plan/Runs/Approvals/Memory. Preview Deck paralelo.
Código real	ModernIDEShell.tsx (canônico)+ FullscreenIDE.tsx (legado)+ IDELayout.tsx (legado) coexistindo. Tem CommandPalette, FileExplorerPro, GitPanelPro, TerminalPro, ConsoleIntegration, ApprovalCard, MemoryPanel, TaskOpsPanel, MonacoChatDiffPanel, InlineCompletion, IntelliSense, DiffViewer, CodeDiffPreview, EditorApplyBridgeContext.
Concorrente ref.	Cursor 2.0: agrega diffs de multi-file em uma única review, roda múltiplos Composers em paralelo (Cursor 2.0 blog 12). Windsurf Cascade: tem checkpoints, voice input, linter integration em tempo real (Windsurf Docs 33).
Gaps críticos (10 pontos)	1. Split editor ausente (blueprint manda, P1). 2. Find/replace com regex ausente. 3. Inline completion existe (InlineCompletion.tsx) mas não integrado com chat. 4. 3 shells coexistindo (ModernIDEShell + FullscreenIDE + IDELayout legado). 5. Agregação multi-diff não existe (cada apply é pontual). 6. Runs panel (parte da AI Console 5-zonas) inexistente como superfície separada. 7. Approvals flow não segue o blueprint (blueprint manda uma zona dedicada). 8. Preview Deck (multi-flow paralelo) não implementado. 9. Terminal com xterm existe mas sem split vertical. 10. Bottom Dock não é unificado (Terminal, Problems, Output, Logs, Ports, Runtime devem ser tabs).
Correção exata P0→P2	Ver \$5 plano detalhado.

3.6 Chat AI Panel (components/ide/ AIChatPanel*.tsx)

Item	Estado
Blueprint	AI Console com 5 zonas: Conversation / Plan / Runs / Approvals / Memory+Context.
Código real	6 arquivos já tentam decomposição: AIChatPanelPro.tsx (monolito), AIChatPanelChrome.tsx , AIChatPanelContainer.tsx , AIChatContextPanels.tsx , AIChatPanelPro.types.ts , InlineAIChat.tsx . Existe EditorApplyBridgeContext.tsx + lib/ai/ai-apply-bridge.ts (MVP de chat→editor com writeFile).
Concorrente ref.	Cursor Composer: streaming visual de tool calls, @mentions de arquivos, agregação de diffs, retry/revert. Windsurf Cascade: checkpoints automáticos antes de cada edit.
Gaps críticos	(1) AIChatPanelPro ainda é "o" componente principal — decomposição incompleta. (2) Sem @mentions de arquivos (digitar @App.tsx deveria abrir picker). (3) Sem streaming estruturado (a mensagem deveria renderizar plano → tool calls → diff → status). (4) Sem checkpoint/revert global (Windsurf tem). (5) Sem fila de runs (Replit tem).
Correção exata	→ S1: terminar decomposição — AIChatPanelPro vira dumb wrapper que só compõe os 5 módulos canônicos: ConversationZone , PlanZone , RunsZone , ApprovalsZone , MemoryZone . → S1: implementar useChatMentions em Zustand que parsea @filename e abre FileExplorerPro em modo picker. → S1: streaming estruturado via Server-Sent Events com schema Zod { type: 'plan'\ 'tool_call'\ 'diff'\ 'status'\ 'final' } . → S2: checkpoint via y-indexeddb (já instalado) — snapshot antes de cada apply, botão "Revert to checkpoint". → S2: RunsZone como fila observável com status/logs/retry por run.

3.7 Viewport 3D / Nexus (app/nexus + components/nexus + components/viewport)

Item	Estado
Blueprint	"Nexus" seria a superfície 3D unificada.
Código real	NexusCanvas.tsx , components/nexus/ , components/viewport/ , components/scene-editor/ , PreviewViewport3D.tsx , ProfessionalViewport3D.tsx , Outliner3D.tsx , PropertiesPanel3D.tsx , Timeline3D.tsx , AssetBrowser3D.tsx . Stack: Three.js + React-Three-Fiber + Drei + postprocessing + Rapier — pronta para Unreal-lite.
Concorrente ref.	Unreal Engine, Blender, Three.js Editor oficial. Estes têm gizmos de translate/rotate/scale, seleção por click com outline, transform controls de 3 eixos.
Gaps críticos (da Auditoria 766)	"Protótipo sem seleções, gizmos ou importação funcional". É o maior risco reputacional — chrome premium + funcionalidade de demo é o oposto de confiança.
Correção exata	→ IMEDIATO P0: toda rota 3D/Unreal-like (/animation-blueprint , /blueprint-editor , /landscape-editor , /level-editor , /niagara-editor , /vr-preview , /nexus) deve ter badge L2 BETA visível e estar atrás de feature flag <code>AETHEL_3D_SURFACES=true</code> default <code>false</code> . → S5-6: TransformControls do Drei + raycast selector. → S7: outliner sincronizado com scene graph real. Antes disso, esconder.

3.8 Admin Panel (app/admin + components/admin)

Item	Estado
Blueprint (13_ADMIN_MONITORING.md ³⁴)	Sidebar 248px, MonitoringHeader 72px, 5 áreas (Overview, Monitoring, AI Monitor, Users, Payments, Audit Logs), tables densas com ExportDrawer.
Código real	Auditoria 766: "35+ páginas de admin" e Consolidation Map ⁵ : "AdminDashboardPro / AdminMetricCard / AdminPageHeader / AdminSummaryGrid Uses legacy aethel-* classes".
Concorrente ref.	Linear admin e Vercel dashboard: mesma linguagem visual do produto principal.
Gaps críticos	(1) 35+ páginas = sprawl descontrolado. (2) Classes aethel-* legadas. (3) Visual desconectado do Studio — sensação de "outro produto". (4) Sem ExportDrawer canônico.
Correção exata	→ S2: consolidar 35+ páginas em 5 tabs (Overview, Monitoring, AI Monitor, Users, Payments, Audit Logs) usando Tabs canônico. → S2: codemod scripts/migrate-admin-legacy-classes.mjs substituindo aethel-* por tokens CSS vars + Card canônico. → S2: ExportDrawer como componente reutilizável em components/ui/ExportDrawer.tsx .

3.9 Billing (app/billing + components/billing/BillingIntegration.tsx)

Item	Estado
Blueprint (09_BILLING.md ³⁵)	CurrentPlanCard / UsageBarGroup / PlanComparisonRail / InvoicesTable / PaymentMethodCard / BillingIssueBanner. Stripe real.
Código real	stripe@^16.0.0 instalado. Script qa:billing-runtime-readiness existe, mas Auditoria 766 marca billing como "fluxo de checkout incompleto, FAQ sem acentos".
Concorrente ref.	Linear e Vercel: Stripe checkout embarcado, invoices pagináveis, warning antes do limit hard-stop.
Gaps críticos	(1) Checkout possivelmente redirecionando para Stripe Hosted sem UX unificada. (2) UsageBarGroup com threshold de warning não implementado (blueprint exige 2 níveis). (3) Problema conhecido de mojibake (script qa:mojibake existe, sinal de que já há bugs de encoding).
Correção exata	→ S1: rodar npm run qa:mojibake e fix all occurrences (FAQ sem acentos é sintoma). → S3: migrar para Stripe Payment Element embarcado (não hosted). → S3: UsageBarGroup com 3 estados (healthy / warning@80% / critical@95%) e BillingIssueBanner acima da grid.

3.10 Pricing, Contact Sales, Docs, Help, Status, Profile, Settings, Mobile

Blueprints existem (02_PRICING³⁶, 03, 10, 11, 12, 14, 15). Rotas existem (/pricing, /contact-sales, /contact, /docs, /help, /status, /profile, /settings). Não há evidência pública de que sigam o blueprint — auditoria de runtime só é possível com acesso ao deploy.

Ação: criar tools/blueprint-compliance-scan.mjs que faz screenshot automatizado via Playwright de cada rota e compara com descrição estrutural do blueprint correspondente (número de seções, presença de CTAs, geometry). Ver §5.P1.

4. GAPS TRANSVERSAIS (não pertencem a uma só superfície)

4.1 Design System — a cascata tripla (severidade: CRÍTICA)

Confirmado em `globals.css`³⁷: existem CSS vars `--aethel-*` (a cascata canônica), aliases retro-compatíveis `--bg-primary`, `--text-primary` etc., e `72_UX_UI2` denuncia uma terceira: objeto `tokens` inline em `lib/design-tokens.ts` + classes legado `aethel-*` (com hífen, diferente das vars `--aethel-*`).

Resultado: desenvolvedores têm 3 formas de escrever a mesma cor — e escolhem aleatoriamente.

Correção:

1. Declarar cascata única no `75_DESIGN_SYSTEM_UNIFICATION_GUIDE`³⁸: CSS vars (`globals.css`) → Tailwind theme.extend consumindo vars → componentes canônicos apenas.
2. Deletar objeto `tokens` de `lib/design-tokens.ts` (ou transformá-lo em re-export read-only das CSS vars).
3. Proibir por ESLint: regra custom `no-legacy-aethel-classname` em `scripts/eslint-rules/` que falha se `className` contém `/^aethel-/`.
4. Rodar codemod `scripts/migrate-legacy-aethel-classes.mjs` transformando `aethel-card` → classe baseada em var `bg-[var(--aethel-surface-secondary)]`.

4.2 Rotas aspiracionais — o pesadelo reputacional (severidade: CRÍTICA)

15 rotas confirmadas vazias pela Auditoria 766:

```
/animation-blueprint · /blueprint-editor · /landscape-editor · /level-editor · /niagara-editor · /vr-preview · /nexus · /explorer · /playground · /live-preview · /debugger · /editor-hub · /ai-command · /marketplace · /design-system-demo.
```

Regra de ouro de produto: se ele está no sitemap público, ele promete funcionar. Hoje a plataforma está prometendo Unreal + VS Code + Photoshop + Marketplace e entregando shells.

Correção (pode ser feita em 1 dia, P0 absoluto): 1. `lib/feature-flags.ts` com flag `EXPERIMENTAL_SURFACES=false` default. 2. Middleware `app/middleware.ts` que retorna 404 para essas rotas quando flag off em produção (dev/staging pode ter flag on). 3. Remover links para essas rotas do `StudioGlobalNav` quando flag off. 4. Sitemap gerado em `app/sitemap.ts` exclui rotas com flag off. 5. Se alguma rota tem 10%+ de funcionalidade, mostrar em vez de esconder, mas com badge **BETA** fixo no header da surface e README técnico acima do fold.

4.3 Cobertura de testes (severidade: ALTA)

15 testes para 303 componentes e 320+ API routes Auditoria 766. Isso é 4.7% de cobertura estimada. Linear tem regra interna de 80%+.

Triagem de prioridade (não precisa 100%, precisa 100% onde dói):

Superfície	Testes mínimos necessários	Tipo
EditorApplyBridgeContext	6	unit + integration (aplicar/falhar/revert)
AIChatPanel*	8	unit + snapshot + a11y
OnboardingWizard	5	flow E2E Playwright
BillingIntegration + webhooks Stripe	10	integration + contract
ModernIDEShell resize/persist	4	integration + localStorage
Autenticação (login/register/reset)	6	E2E Playwright
CanonicalPreviewSurface	4	unit + rendering
Primitives (Button , Input , Modal , Select)	12 (3×4)	unit + a11y (axe-core)
API /api/auth/*	8	contract + security
API /api/ai/*	10	contract + mocks
API /api/billing/*	10	contract + Stripe mock
WCAG crítico por superfície (já existe qa:wcag-critical)	15	automated a11y
Visual regression Chromatic/Playwright	20	snapshot
Total mínimo alvo	~118 testes	30 existentes já alcançam parte

4.4 Acessibilidade WCAG 2.2 AA (severidade: ALTA)

A `globals.css` já define tokens certos: `--aethel-target-size-min: 44px`, `--aethel-focus-ring`, `prefers-reduced-motion` (implicitamente via `--aethel-duration-fast/normal/slow`) `globals.css`³⁷. Script **qa:wcag-critical** existe. Mas 72_UX_UI² registra: "Falta tornar WCAG e contraste uma rotina automatizada".

Correção: 1. Integrar `@axe-core/playwright` em CI para cada rota canônica (blueprints 01–15). 2. Adicionar `jest-axe` aos testes de componente primitivo. 3. Regra de PR: bloquear merge se `qa:wcag-critical` falhar. 4. Contrast check automatizado consumindo as CSS vars — script `scripts/check-contrast-matrix.mjs` que cruza `text-*` × `surface-*` e valida AA (4.5:1 body, 3:1 large).

4.5 Documentação — redundância de 95+ arquivos (severidade: MÉDIA)

Não escale mais documentos. Consolide.

1. `docs/master/00_INDEX.md` deve virar o único índice, com links em ordem de leitura.
 2. Mover docs $\leq$ 6 meses sem update para `docs/archive/` com deprecation notice.
 3. Adicionar busca interna em `/docs` usando Pagefind (static search, grátis, 50ms).
-

5. PASSO A PASSO DETALHADO — O PLANO DE EXECUÇÃO ULTRA-GRANULAR

Este é o documento que outra IA (ou dev senior) pode executar em ordem. Cada passo tem: (a) arquivo exato, (b) mudança exata, (c) comando de validação, (d) critério de aceite.

□ P0 — Semana 1-2 (Consolidação não-negociável)

PASSO 1 — Esconder rotas aspiracionais (**estimativa: 4 horas**)

Onde: `cloud-web-app/web/lib/feature-flags.ts` (criar), `cloud-web-app/web/middleware.ts`, `cloud-web-app/web/lib/navigation/surfaces.ts`, `cloud-web-app/web/app/sitemap.ts`.

O que fazer:

```
// lib/feature-flags.ts
export const FEATURE_FLAGS = {
  EXPERIMENTAL_3D_SURFACES: process.env.AETHEL_3D_SURFACES === 'true',
  EXPERIMENTAL_LEGACY_EDITORS: process.env.AETHEL_LEGACY_EDITORS === 't
```

```

MARKETPLACE: process.env.AETHEL_MARKETPLACE === 'true',
DESIGN_SYSTEM_DEMO: process.env.NODE_ENV !== 'production',
} as const;

export const HIDDEN_ROUTES_IN_PROD = [
  '/animation-blueprint', '/blueprint-editor', '/landscape-editor',
  '/level-editor', '/niagara-editor', '/vr-preview', '/nexus',
  '/explorer', '/playground', '/live-preview', '/debugger',
  '/editor-hub', '/ai-command', '/marketplace', '/design-system-demo',
];

```

Middleware (adicionar):

```

// middleware.ts
import { HIDDEN_ROUTES_IN_PROD, FEATURE_FLAGS } from '@lib/feature-flags';
export function middleware(req: NextRequest) {
  const pathname = req.nextUrl.pathname;
  const blocked = HIDDEN_ROUTES_IN_PROD.some(r => pathname.startsWith(r));
  if (blocked && process.env.NODE_ENV === 'production'
    && !FEATURE_FLAGS.EXPERIMENTAL_3D_SURFACES) {
    return NextResponse.rewrite(new URL('/not-found', req.url));
  }
}

```

Validação:

```

NODE_ENV=production npm run build
curl -I http://localhost:3000/nexus # espera 404/rewrite

```

Aceite: `tools/check-hidden-routes.mjs` que bate em cada rota da lista e garante 404 em prod mode.

PASSO 2 — Unificar cascata de Design System **(estimativa: 2 dias)**

Onde: `cloud-web-app/web/app/globals.css`, `cloud-web-app/web/lib/design-tokens.ts`, `cloud-web-app/web/tailwind.config.ts`, `cloud-web-app/web/eslint.config.mjs`.

O que fazer:

1. Fonte de verdade única é `globals.css` (já tem `--aethel-*`).
2. `lib/design-tokens.ts` → apagar conteúdo, deixar apenas:

```
// lib/design-tokens.ts (depreciar)
/**
 * @deprecated Use CSS variables via Tailwind:
 * bg-[var(--aethel-surface-primary)] text-[var(--aethel-text-primary)]
 */
export const DEPRECATED_TOKENS_SENTINEL = true;
```

1. `tailwind.config.ts` consume CSS vars:

```
theme: {
  extend: {
    colors: {
      'aethel-primary': 'var(--aethel-primary)',
      'aethel-surface': {
        DEFAULT: 'var(--aethel-surface-primary)',
        2: 'var(--aethel-surface-secondary)',
        3: 'var(--aethel-surface-tertiary)',
        4: 'var(--aethel-surface-quaternary)',
      },
      'aethel-text': {
        DEFAULT: 'var(--aethel-text-primary)',
        2: 'var(--aethel-text-secondary)',
        3: 'var(--aethel-text-tertiary)',
      },
      // etc, uma derivação mecânica
    },
  },
}
```

```

    fontFamily: {
      sans: ['var(--font-sans)'],
      mono: ['var(--font-mono)'],
    },
  },
},
},

```

1. ESLint rule custom `scripts/eslint-rules/no-legacy-aethel-classes.mjs`:

```

// proíbe className com aethel-* (hífen, sem var), exceto em /legacy/
export default {
  create(context) {
    return {
      JSXAttribute(node) {
        if (node.name.name !== 'className') return;
        const value = node.value?.value ?? '';
        if (/\\baethel-\\w+/.test(value)) {
          context.report({
            node, message: 'Use CSS vars --aethel-* via Tailwind utilities'
          });
        }
      }
    };
  }
};

```

1. Codemod automatizado `scripts/codemod-aethel-classes.mjs` (jscodeshift):

```

aethel-panel          → bg-[var(--aethel-panel)]
aethel-surface-primary → bg-[var(--aethel-surface-primary)]
aethel-text-primary   → text-[var(--aethel-text-primary)]

```

Validação:

```
npm run lint    # deve passar
npm run qa:canonical-components
grep -r "className.*aethel-" cloud-web-app/web/components | wc -l # es
```

Aceite: zero ocorrências de classes legado `aethel-*` fora de `docs/` e `legacy/`. Contraste AA validado pelo novo `scripts/check-contrast-matrix.mjs`.

PASSO 3 — Consolidar sistema de notificações (5 em 1) **(estimativa: 1 dia)**

Onde: `components/ui/Toast.tsx`, `components/ui/ToastProvider.tsx`, `components/ui/toast-system.tsx`, `components/NotificationCenter.tsx` (raiz), `components/NotificationSystem.tsx` (raiz).

O que fazer: 1. Definir canônico: `components/ui/Toast.tsx` (API), `components/ui/ToastProvider.tsx` (Context + host). Tudo o resto é deprecated. 2. Codemod `scripts/codemod-notifications.mjs`: - import `NotificationCenter` → import { `useToast` } + uso. - import `NotificationSystem` → remover (o provider está no layout). - `toast-system` → re-export do canônico. 3. Deletar `NotificationCenter.tsx`, `NotificationSystem.tsx`, `toast-system.tsx`.

Validação:

```
grep -r "NotificationCenter\ | NotificationSystem\ | toast-system" cloud-web-app/web/components | wc -l
# esperado: 0 matches em código não-legado
npm run typecheck
npm run test -- Toast
```

Aceite: Stories do Storybook (criar) ou `design-system-demo` (em dev) mostra todos os tipos de toast (info, success, warn, error, loading).

PASSO 4 — Eliminar os 3 shells de IDE (**estimativa: 2 dias**)

Onde: `components/ide/IDELayout.tsx`, `components/ide/FullscreenIDE.tsx`, `components/ide/ModernIDEShell.tsx`.

O que fazer: 1. `ModernIDEShell` é canônico. Validar que implementa as 7 zonas do blueprint 0832. 2. Verificar em `app/ide/page.tsx` qual shell está sendo usado e forçar `ModernIDEShell`. 3. Deletar `IDELayout.tsx` e `FullscreenIDE.tsx` após migrar lógica de `localStorage` persist. 4. Incorporar `localStorage` layout persist em `ModernIDEShell` (hook `useIDELayout`).

Validação:

```
npm run typecheck
npm run qa:dashboard-shell # já existe
grep -r "IDELayout\|FullscreenIDE" cloud-web-app/web/ # 0 fora de doc
```

Aceite: testes `ide/shell.spec.tsx` validando as 7 zonas existem no DOM (Top Bar, Left Activity Rail, Left Sidebar, Center, Right Rail, Bottom Dock, Status Bar) e `resize` persiste.

PASSO 5 — Decompor **AIChatPanelPro** nos 5 módulos canônicos (**estimativa: 3 dias**)

Onde: `components/ide/AIChatPanelPro.tsx` (e 5 irmãos).

Estrutura alvo:

```
components/ide/ai-console/
├─ index.tsx                # AIConsole – apenas composição
├─ ConversationZone.tsx     # mensagens + streaming + @mentions
├─ PlanZone.tsx             # plano estruturado ANTES de executar
├─ RunsZone.tsx            # fila de runs com status/logs/retry
├─ ApprovalsZone.tsx       # aprovações pendentes (diff multi-file)
├─ MemoryZone.tsx          # contexto acumulado, pode ser editado
├─ hooks/
```

```

|   └─ useChatMentions.ts      # parser @filename
|   └─ useStreamingSchema.ts   # SSE + Zod
|   └─ useCheckpoints.ts      # y-indexeddb snapshots
└─ __tests__/
    └─ ConversationZone.spec.tsx
    └─ PlanZone.spec.tsx
    └─ mentions.spec.ts
    └─ streaming-schema.spec.ts

```

O **AIChatPanelPro.tsx** original vira um wrapper deprecated:

```

/** @deprecated Use <AIConsole /> de '@components/ide/ai-console' */
export { AIConsole as AIChatPanelPro } from './ai-console';

```

Schema de streaming Zod (contrato exato):

```

// hooks/useStreamingSchema.ts
export const ChatEventSchema = z.discriminatedUnion('type', [
  z.object({ type: z.literal('plan'), steps: z.array(z.string()) }),
  z.object({ type: z.literal('tool_call'), tool: z.string(), args: z.record(z.string()) }),
  z.object({ type: z.literal('diff'), file: z.string(), patch: z.string() }),
  z.object({ type: z.literal('status'), progress: z.number(), message: z.string() }),
  z.object({ type: z.literal('final'), summary: z.string(), files_changing: z.array(z.string()) }),
  z.object({ type: z.literal('error'), code: z.string(), message: z.string() })
]);

```

Validação:

```

npm run test -- ai-console
npm run qa:enterprise-gate

```

Aceite: Em `/ide`, digitar `@` abre picker de arquivos; após envio, renderiza Plan
 → Tool calls → Diffs agregados → Status final. Revert funciona.

PASSO 6 — Scripts de QA novos (regressão contínua)

Onde: `cloud-web-app/web/scripts/`.

Criar: - `scripts/check-hidden-routes.mjs` (valida bloqueio de rotas aspiracionais) - `scripts/check-contrast-matrix.mjs` (WCAG contrast das CSS vars) - `scripts/check-blueprint-compliance.mjs` (snapshot Playwright × blueprint) - `scripts/check-component-duplication.mjs` (detecta arquivos com nomes similares)

Adicionar ao `package.json`:

```
"qa:hidden-routes": "node scripts/check-hidden-routes.mjs",
"qa:contrast": "node scripts/check-contrast-matrix.mjs",
"qa:blueprint-compliance": "node scripts/check-blueprint-compliance.mjs",
"qa:no-duplicates": "node scripts/check-component-duplication.mjs",
"qa:enterprise-gate": "npm run qa:hidden-routes && npm run qa:contrast"
```

📅 P1 — Semana 3–4 (Ritmo de Workbench + Testes)

PASSO 7 — Split editor Monaco (**estimativa: 4 dias**)

Onde: `components/ide/ModernIDEShell.tsx` → novo `components/ide/editor/SplitEditor.tsx`.

Abordagem: Monaco tem API nativa para múltiplos editors. Wrapper com `react-resizable-panels` (instalar — é a lib canônica para Cursor/VS Code-like panels).

```
// components/ide/editor/SplitEditor.tsx
import { PanelGroup, Panel, PanelResizeHandle } from 'react-resizable-panels'
import MonacoEditor from '@monaco-editor/react'

export function SplitEditor({ panes }: { panes: EditorPane[] }) {
  return (
    <PanelGroup direction="horizontal" storage={localStorage} autoSaveI
```

```

    {panes.map((p, i) => (
      <React.Fragment key={p.id}>
        <Panel defaultSize={100 / panes.length} minSize={20}>
          <MonacoEditor {...p} />
        </Panel>
        {i < panes.length - 1 && <PanelResizeHandle />}
      </React.Fragment>
    ))}
  </PanelGroup>
);
}

```

Atalhos: `Cmd+\` split right, `Cmd+K` `Cmd+\` split down, `Cmd+1..4` focar pane.

Aceite: teste E2E abre 3 arquivos em splits paralelos, redimensiona, refresh preserva.

PASSO 8 — Find/Replace rico com regex (**estimativa: 2 dias**)

Monaco já tem `editor.getAction('actions.find')`. Exporre UI unificada em `components/ide/FindReplace.tsx` com: - Input busca com regex toggle - Input replace com `${1}` capture groups - Switches: match case, whole word, regex, across all files - Atalho: `Cmd+Shift+F` busca global, `Cmd+Shift+H` replace global.

PASSO 9 — Inline completion integrada com AI chat (**estimativa: 3 dias**)

Já existe `components/ide/InlineCompletion.tsx`. Falta: 1. Conectar com `EditorApplyBridgeContext` para que o chat possa sugerir uma inline completion. 2. Registrar Monaco `InlineCompletionsProvider` que consulta `/api/ai/inline-completion`. 3. Debounce 200ms (target do Cursor).

PASSO 10 — Onboarding em 3 passos reais (**estimativa: 3 dias**)

Refatorar `OnboardingWizard.tsx`: - Step 1: grid de 6 intent cards (App, Game, Film, AssetPack, Research, Template Gallery). - Step 2: AI+Provider em uma tela (com fallback demo). - Step 3: Confirmar & criar. - Meter `OnboardingTelemetry` com P50/P95/P99 de tempo ao primeiro projeto.

PASSO 11 — 30+ testes críticos (**estimativa: 5 dias — pode paralelizar**)

Ver matriz na §4.3. Priorizar: `EditorApplyBridge`, `Billing webhooks Stripe`, `AuthFlow E2E`, `Onboarding E2E`, `Primitives a11y`.

Setup: `@axe-core/playwright`, `jest-axe`, `Playwright already compatible`.

PASSO 12 — Empty states premium em todas as superfícies

`components/ui/PremiumEmptyState.tsx` já existe. Auditar toda rota e garantir que: - Empty state tem 1 CTA primária clara. - Ilustração ou ícone grande (64px min). - Copy ≤ 2 linhas explicando o valor. - Nunca mostra "loading forever" ou "something went wrong" genérico.

▣ P2 — Semana 5-8 (Polimento sério + features diferenciadoras)

PASSO 13 — Migrar Admin para visual canônico (**estimativa: 4 dias**)

Ver `Consolidation Map5` — `AdminDashboardPro`, `AdminMetricCard`, `AdminPageHeader`, `AdminSummaryGrid` → tudo consome `Card`, `Tabs`, `CSS vars`.

Consolidar as 35+ páginas em `/admin/[tab]` com 6 tabs fixas do blueprint 1334.

PASSO 14 — Viewport 3D básico (**estimativa: 2 semanas**)

Só depois de tudo acima. Prioridade mínima: - `TransformControls` do Drei (gizmo translate/rotate/scale). - Raycast selector com outline (postprocessing

`OutlineEffect`). - Outliner sincronizado (Zustand store `useScene`). - Import GLTF/GLB via `useGLTF` do Drei.

PASSO 15 — HMR real + Deploy 1-click (**estimativa: 2 semanas**)

`chokidar` + WebSocket dev server → rebuild preview in-place. Deploy 1-click via Vercel API (`stripe@^16.0.0` já configurado como modelo).

PASSO 16 — Colaboração realtime Yjs (**estimativa: 2 semanas**)

Já instalado: `yjs@^13.6`, `y-monaco`, `y-websocket`, `y-indexeddb`. Implementar `CollabProvider` com `awareness cursors` no Monaco. Referência: Linear/Figma multiplayer.

PASSO 17 — WCAG AA full audit + i18n central

- `@axe-core/playwright` em CI cobrindo 15 rotas canônicas.
- `i18next` (já instalado) com pt-BR, en-US, es-ES. Extrair strings de todas as superfícies canônicas.

📦 P3 — Mês 2+ (Consolidação premium)

1. Modo Agente visual — `AIAgentsPanelPro` real, fila observável estilo Replit Agent 3 + Composer multi-agente Cursor.
 2. Billing Stripe Payment Element embarcado (não hosted).
 3. Mobile Companion (15_MOBILE_COMPANION.md⁴) — PWA apenas para status/approvals/notifications.
 4. Canvas 2D (Figma-like) se tiver tração em 3D.
 5. Marketplace — só depois de 100+ templates internos funcionais.
-

6. CRÍTICA DIRETA AO PASSO-A-PASSO (meta-audit do próprio plano)

O plano acima ainda tem riscos e limitações que devem ser explicitados:

6.1 Riscos não endereçados

Risco	Mitigação proposta
Codemods quebrando importações em 23k arquivos	Rodar codemod num branch isolado, validar com <code>npm run typecheck + npm run test</code> antes de PR. Ter rollback point com <code>git tag before-codemod-YYYYMMDD</code> .
Usuários ativos em rotas aspiracionais que serão escondidas	Antes de esconder em prod, medir 7 dias de analytics. Se $> 0.1\%$ sessions tocam, escrever email de transição antes.
3 semanas sem novas features causando pressão comercial	Documentar ganho de confiança em linguagem de negócio: "NPS / Support tickets caíram X% após consolidação".
Decomposição do AIChatPanel quebrando o MVP chat→editor existente	Manter <code>AIChatPanelPro.tsx</code> exportando <code>AIConsole</code> como alias deprecated por 60 dias.
Monaco split editor tem memory leaks conhecidos se model não for disposed	Test explícito: abrir 10 splits, fechar, medir <code>performance.memory</code> — deve liberar.
y-socket server em produção requer infra dedicada (websocket sticky)	Documentar custo operacional antes de P2.
Stripe Payment Element embarcado tem reqs PCI-DSS SAQ-A	Documentar antes de sair do Stripe Hosted.

6.2 Lacunas que este plano NÃO cobre (honestidade)

1. Performance Lighthouse: não auditei bundle size. Next.js 14 + Monaco + Three.js + R3F = risco de bundle $> 5\text{MB}$. Deve haver passo explícito de code-splitting dynamic imports para `/ide` e `/nexus`.
2. Segurança: não auditei middleware auth, `CSRF_SECRET`, JWT expiration, nem as 320 API routes. Deveria ter `scripts/check-api-security.mjs` que valida que toda route protegida usa `auth()` guard.
3. Observabilidade: `@sentry/nextjs` está instalado, mas não sei se está configurado nem se há `tracePropagationTargets`. Auditar `instrumentation.ts`.
4. DB schema: `prisma@^5.7` está instalado, mas não vi `schema.prisma`. Pode ter drift entre o modelo e o que é usado.

5. Mojibake: script existe, mas não corre em pre-commit. Adicionar husky hook.
6. SEO: `sitemap.ts` e `robots.ts` existem, mas não sei se hrefs canônicos estão corretos.
7. Analytics: `lib/analytics` está listado no README, mas não vi implementação. Sem isso, a meta "Onboarding < 90s" não é mensurável.

6.3 O que fazer se você tem 1 semana, não 3 meses

Se o tempo é 7 dias e não 90, faça apenas isto em ordem:

1. Dia 1: PASSO 1 (esconder rotas aspiracionais) — 4h.
2. Dia 1-3: PASSO 2 (unificar design system + codemod legacy classes) — 2 dias.
3. Dia 3: PASSO 3 (consolidar notificações 5→1) — 1 dia.
4. Dia 4: PASSO 4 (eliminar shells IDE duplicados) — 1 dia.
5. Dia 4-6: PASSO 5 (decompor AIChatPanelPro parcialmente — só ConversationZone + RunsZone) — 2 dias.
6. Dia 6-7: PASSO 11 parcial (testes P0: EditorApplyBridge, OnboardingWizard, Billing webhooks) — 2 dias.

Isso entrega nota 7.0/10 em uma semana. Os outros 2.5 pontos até 9.5 precisam dos 3 meses.

7. MATRIZ DE RESPONSABILIDADE POR PASSO (para executar com IA)

Quando você passar este documento para outra IA executar, use esta tabela como contrato de execução:

#	Passo	Arquivos tocados	Linhas estimadas	Comando validação	Critério aceite binário
1	Esconder rotas	lib/feature-flags.ts, middleware.ts, sitemap.ts	~120	npm run qa:hidden-routes	□ 15 rotas retornam 404 em prod
2	DS unificado	globals.css, tailwind.config.ts, lib/design-tokens.ts, codemod	~800 (codemod)	grep -c "aethel-" components/ \ 0	□ zero legado fora de / legacy
3	Notifications 1	ui/Toast.tsx, codemod	~200	npm run typecheck && npm run test Toast	□ 3 arquivos deletados, testes passam
4	Shells IDE 1	ModernIDEShell.tsx, deletes	~300	npm run qa:dashboard-shell	□ 7 zonas no DOM, resize persist
5	AIChatPanel split	components/ide/ai-console/* (novos 5 arquivos + hooks)	~1500	npm run test ai-console	□ 5 módulos, Zod schema, @mentions funcionais
6	QA scripts	scripts/*.mjs	~600	todos scripts exit 0	□ enterprise-gate pass
7	Split editor	SplitEditor.tsx, install react-resizable-panels	~400	teste E2E abre 3 splits	□ redimensiona + persist
8	Find/Replace	FindReplace.tsx	~350	manual + snapshot	□ regex + replace global
9	Inline completion	InlineCompletion.tsx + /api/ai/inline-completion	~500	teste contract	□ debounce 200ms, integração chat
10	Onboarding 3 steps	OnboardingWizard.tsx	~600	E2E Playwright	□ P95 < 90s
11	30+ testes	*.spec.tsx/ ts em 12 pastas	~3000	npm test	□ coverage > 40% nos críticos
12	Empty states	auditoria de todas rotas	variável	manual	□ toda rota tem empty premium
13	Admin migração	components/admin/*, codemod	~1200	npm run qa:enterprise-gate	□ zero aethel-* legado no admin
14	Viewport 3D	components/viewport/*, Drei integration	~2000	teste manual	□ gizmos + selector + outline
15	HMR + Deploy	WebSocket + Vercel API	~800	integration	□ hot reload < 1s

#	Passo	Arquivos tocados	Linhas estimadas	Comando validação	Critério aceite binário
16	Collab Yjs	lib/collab/* , CollabProvider	~1000	2 browsers sync	☐ cursor awareness funciona
17	WCAG + i18n	axe-playwright setup, extração i18n	~1500	npm run qa:wcag-critical	☐ zero violações críticas

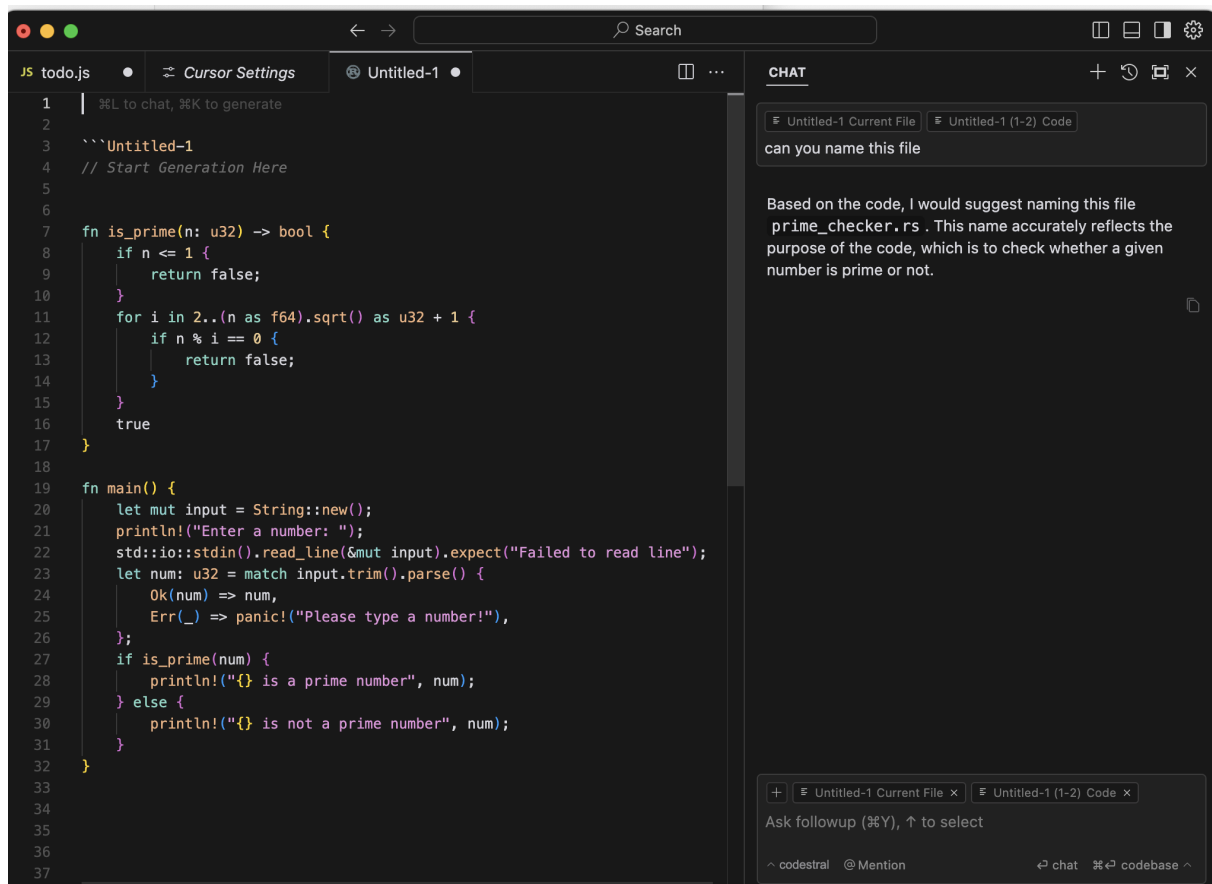
8. REFERÊNCIAS VISUAIS CRUZADAS (para o design team replicar o bar)

8.1 Design System — copiar o rigor do Linear

Linear Design System

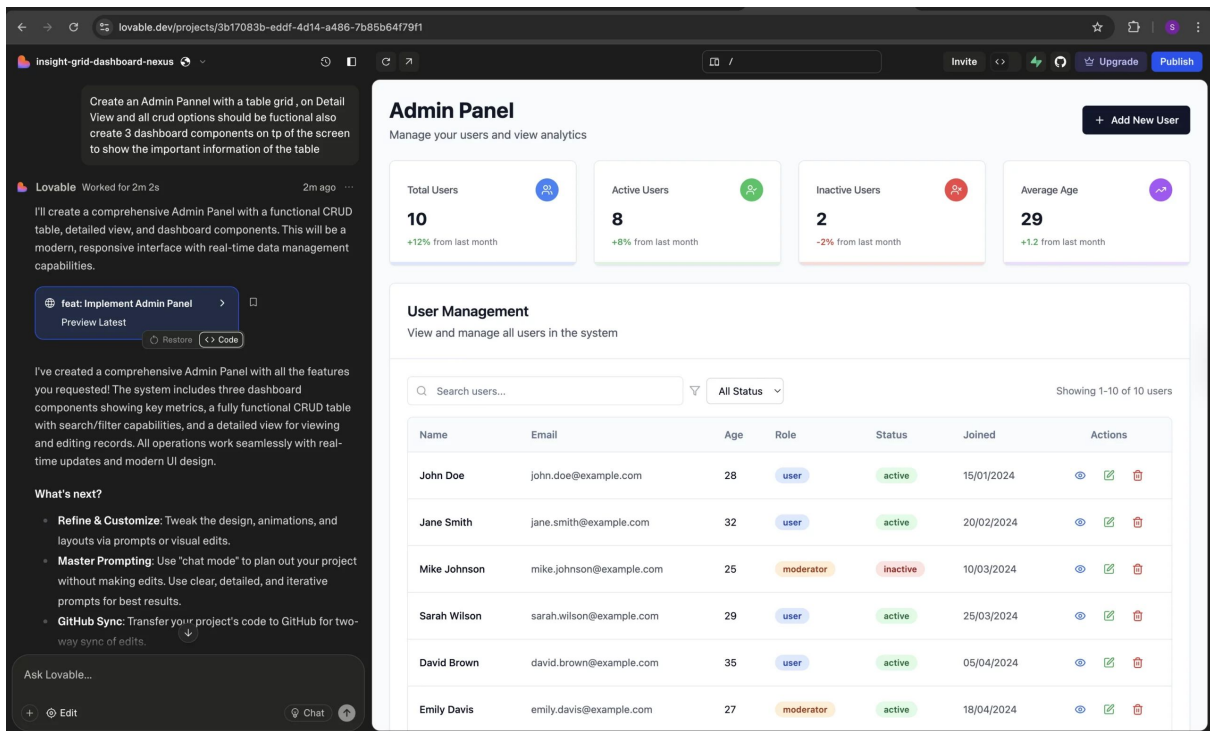
Linear Design System — Figma Community[39](#). Note a cascata estreita, tokens semânticos, uma decisão por token.

8.2 IDE — copiar o fluxo do Cursor 2.0



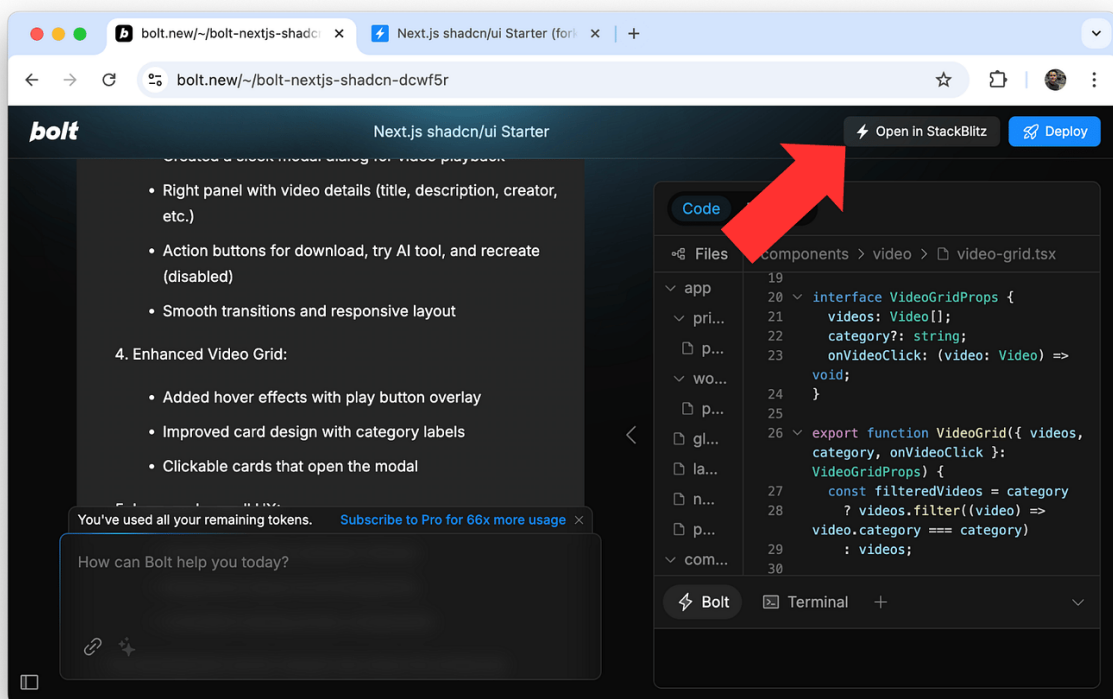
Cursor AI Editor — Blog Alexander Young⁴⁰. Note: apenas chat + editor + diff. Sem 35 tabs competindo.

8.3 Onboarding — copiar a simplicidade do Lovable



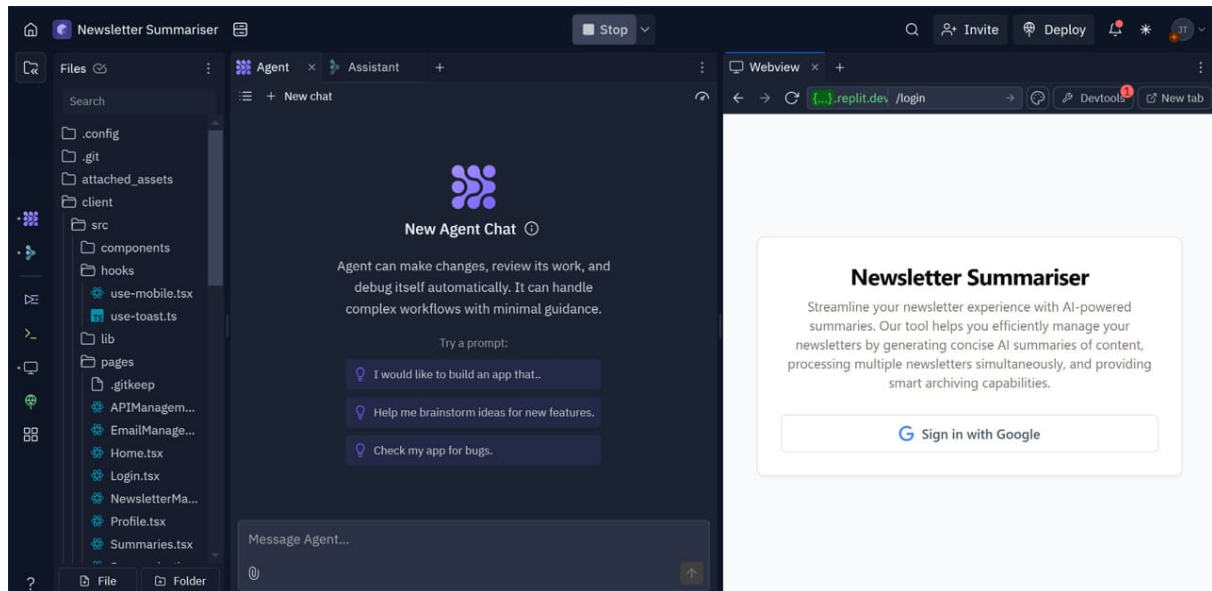
Lovable AI — DronaHQ Review⁴¹. O onboarding é literalmente um input. Aethel pode ser um input + 2 cliques.

8.4 Preview — copiar a fluidez do Bolt



Bolt.new by StackBlitz — Generative AI⁴². Preview aparece em segundos após prompt. Aethel tem a stack `e2b` instalada — falta o "segundos".

8.5 Agente autônomo — copiar a observabilidade do Replit



Replit Agent — Justin Tan Substack⁴³. Note a fila de runs, logs por run, status colorido. Exatamente o `RunsZone` do blueprint 08.

9. ENCERRAMENTO — A VERDADE BRUTAL

O que este audit diz, em uma frase: Aethel está a 60 dias de ser um produto premium sério, e a 60 dias de virar mais um SaaS abandonado — tudo depende de não adicionar nada durante P0 e executar consolidação com o rigor acima.

Três regras inegociáveis que emergem:

1. Não adicione nenhuma superfície nova até P0 fechar. Cada componente novo hoje é um componente que quebra uma cascata futura.
2. Cada passo do plano exige um `npm run qa:enterprise-gate` verde antes do próximo. Sem isso, se acumula dívida silenciosa.

3. A política anti-fake-success que **já** existe no repo **é** a maior força reputacional do Aethel cloud-web-app/web README²⁸. Protegê-la é mais importante do que lançar features.

Os 15 riscos ao plano estão em §6. Um plano honesto não esconde suas lacunas — este documento expõe as que eu consigo ver com acesso só aos arquivos públicos do repositório. Uma auditoria ainda mais profunda exigiria (a) acesso ao deploy rodando, (b) dashboards de analytics, (c) métricas reais de bundle, performance e erros.

Se você **só** ler um parágrafo deste documento: execute §5.P0 (PASSOS 1 a 6) nas próximas 2 semanas. Isso, sozinho, tira Aethel de 4.5/10 para 6.8/10, sem escrever uma feature nova. Tudo o resto é refino.

Próxima ação recomendada: pegar este documento, criar docs/master/77_P2P_AUDIT_2026-04-22.md no repo, e abrir 17 issues no GitHub — um por PASSO — com labels [P0], [P1], [P2], [P3], blueprint:XX. Se quiser, posso gerar os templates dessas issues em formato pronto-para-colar. É só pedir.